

CREACIÓN DEL PROYECTO FRONTEND



✚ Agregar el Componente Formulario y Registrar Personas

🎯 Introducción

En esta lección vamos a crear un nuevo componente llamado `formulario` que nos permitirá **agregar personas** al listado general. También modificaremos la clase `PersonasComponent` para incluir un botón que nos redirija a este formulario, y configuraremos las rutas necesarias.

✅ Paso 1: Crear el componente `formulario`

Abrimos la terminal y dentro del proyecto ejecutamos el siguiente comando:

```
ng g c formulario --skip-tests=true --inline-style=true
```

Esto crea la carpeta `src/app/formulario` con los archivos necesarios para el componente.

✓ Paso 2: Registrar el nuevo componente en el archivo de rutas

Abrimos el archivo:

 `src/app/app.routes.ts`

Y agregamos las siguientes rutas:

```
import { Routes } from '@angular/router';
import { PersonasComponent } from '../personas/personas.component';
import { FormularioComponent } from '../formulario/formulario.component';

export const routes: Routes = [
  { path: '', component: PersonasComponent },
  {path: 'personas', component: PersonasComponent, children:[
    {path: 'agregar', component: FormularioComponent}
  ]}
];
```

✓ Paso 3: Agregar botón para mostrar el formulario e incrustar

`<router-outlet>`

Editamos el archivo:

 `src/app/personas/personas.component.html`

Y agregamos un botón al inicio para navegar al formulario. Además, agregamos `<router-outlet>` para incrustar el componente del formulario, según lo definido en el archivo `app.routes.ts`

```
<div style="text-align: center">
  <button style="cursor: pointer" (click)="irAgregar()">+</button>
</div>
<div class="box">
  @for (persona of personas; track persona.idPersona) {
    <div class="item">
      <ul>
        <li>
          {{ persona.idPersona }} {{ persona.nombre }}
        </li>
      </ul>
    </div>
  }
</div>
```

```

        </li>
      </ul>
    </div>
  }
</div>
<!-- incrusta el componente de formulario -->
<router-outlet></router-outlet>

```



Paso 4: Agregar el método `irAgregar()` en `PersonasComponent`

Abrimos el archivo:

 `src/app/personas/personas.component.ts`

Y agregamos el método `irAgregar()`. También recordamos que este componente usa `<router-outlet>`, por lo tanto, debe importar el módulo de rutas `RouterModule`:

```

import { Component } from '@angular/core';
import { Persona } from '../persona.model';
import { PersonaService } from '../persona-service';
import { ActivatedRoute, Router, RouterModule } from '@angular/router';

@Component({
  selector: 'app-personas',
  imports: [RouterModule],
  templateUrl: './personas.component.html'
})
export class PersonasComponent {
  personas: Persona[] = [];

  constructor(private personaService: PersonaService,
               private router: Router,
               private route: ActivatedRoute
  ) { }

  ngOnInit(): void{
    this.personaService.obtenerPersonas()
      .subscribe(
        (personasObtenidas: Persona[]) => {
          //cargamos los datos de persona obtenidos en el arreglo local
          this.personas = personasObtenidas;
          this.personaService.setPersonas(this.personas);
        }
      );
  }
}

```

```

        console.log('personas obtenidas del subscriber:' + this.personas);
    }
    );
}

irAgregar(){
    console.log('nos vamos a agregar');
    this.router.navigate(['./personas/agregar']);
}

}

```

✓ Paso 5: Crear el formulario HTML

Editamos el archivo:

 src/app/formulario/formulario.component.html

Y agregamos el siguiente código:

```

<div class="box">
  <form class="item" #f="ngForm" (ngSubmit)="f.form.valid && onGuardarPersona()">
    <input type="text" name="nombre" placeholder="Nombre" [(ngModel)]="nombreInput" required/>
    <button type="submit" style="cursor: pointer">+</button>
  </form>
</div>

```

✓ Paso 6: Lógica para guardar persona en `FormularioComponent`

Editamos el archivo:

 src/app/formulario/formulario.component.ts

Y agregamos el método `onGuardarPersona()`. Este método crea una nueva instancia de `Persona` con los datos capturados en el formulario y llama al servicio `PersonaService`.

Además, debido a que estamos utilizando `[(ngModel)]` en el formulario, debemos importar `FormsModule` en este componente standalone.

```
import { Component } from '@angular/core';
```

```

import { PersonaService } from '../persona-service';
import { ActivatedRoute, Router } from '@angular/router';
import { Persona } from '../persona.model';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-formulario',
  imports: [FormsModule],
  templateUrl: './formulario.component.html',
  styles: ``
})
export class FormularioComponent {
  idPersona?: number;
  nombreInput?: string;

  constructor(private personaService: PersonaService,
               private router: Router,
               private route: ActivatedRoute
  ) { }

  ngOnInit() {

  }

  onGuardarPersona(){
    // this.idPersona ?? 0 significa: si idPersona está definido, úsalo; si no, usa 0.
    // this.nombreInput ?? '' evita pasar undefined como nombre, y en su lugar pasaría una cadena vacía.
    const personaAGuardar = new Persona(this.idPersona ?? 0, this.nombreInput ?? '');
    this.personaService.agregarPersona(personaAGuardar);
    this.router.navigate(['personas']);
  }
}

```

✖ ¿Qué significa el signo ? en `idPersona?: number` y `nombreInput?: string`?

La sintaxis con el signo de interrogación (?) en una propiedad de clase en TypeScript indica que **esa propiedad es opcional**.

```

idPersona?: number;
nombreInput?: string;

```

🔍 Esto significa que:

- No es obligatorio asignarles un valor al momento de crear la instancia.

- Su valor puede ser `undefined` si no se establece explícitamente.

✚ ¿Qué significa ?? (Nullish Coalescing Operator) en TypeScript?

El operador ?? se llama **operador de fusión nula** (*nullish coalescing*).

Se usa para asignar un valor **por defecto solo si** el valor original es `null` o `undefined`.

🔍 Explicación línea por línea:

```
this.idPersona ?? 0
```

👉 Si `this.idPersona` es `null` o `undefined`, se usará `0`.

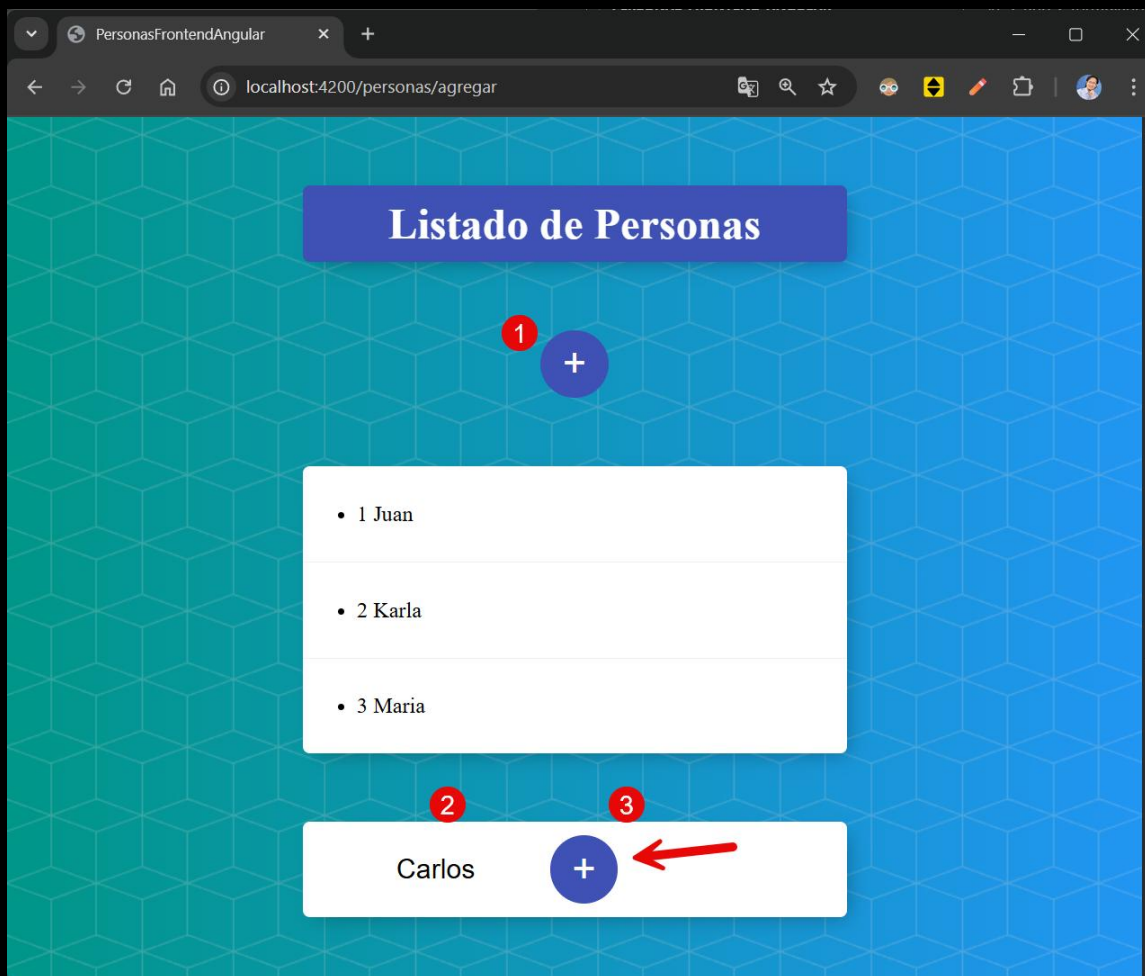
```
this.nombreInput ?? ''
```

👉 Si `this.nombreInput` es `null` o `undefined`, se usará una cadena vacía (`''`).

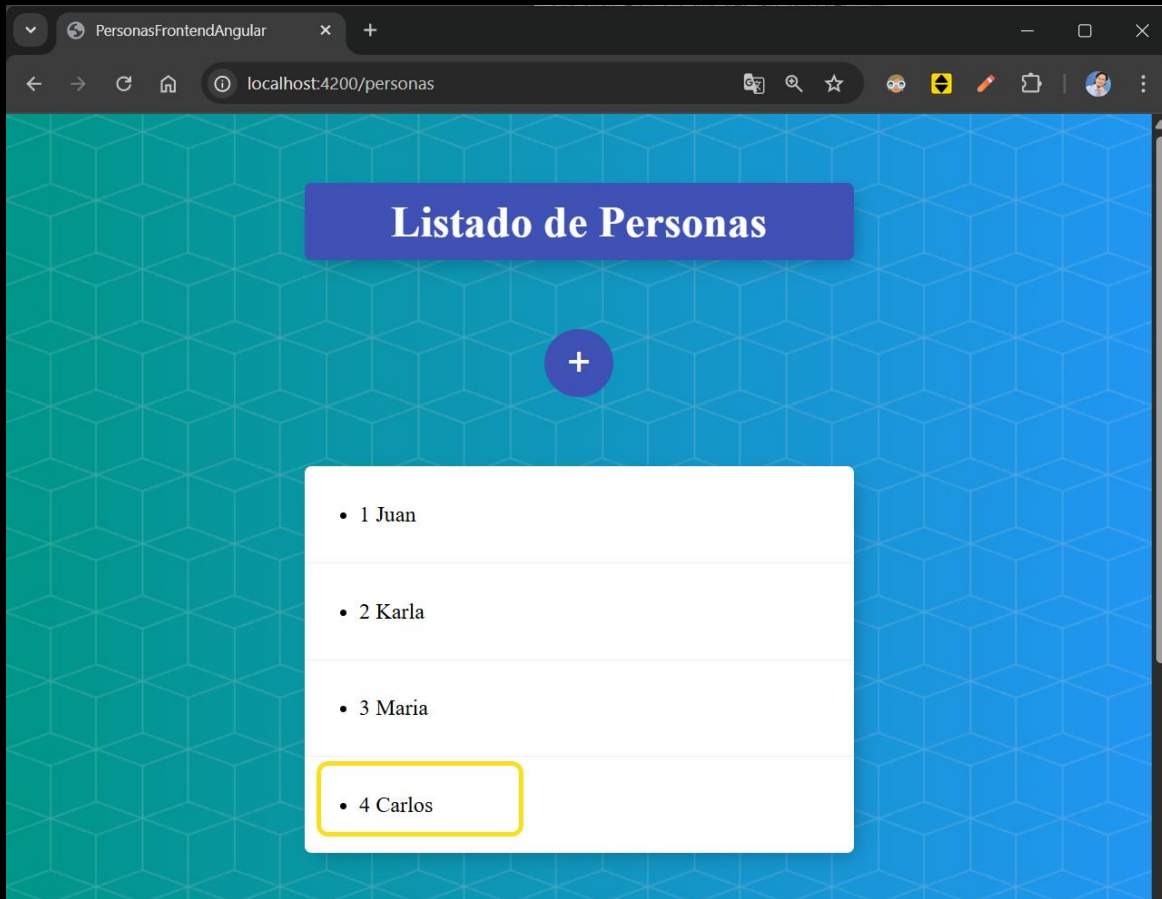
✅ Ejecutamos de la aplicación

Volvemos a ejecutar la aplicación:

- ♦ Detenemos la aplicación, para ello vamos a la consola donde se está ejecutando la aplicación y presionamos `ctl + c`
- ♦ Volvemos a ejecutar el comando: `ng serve -o` para levantar nuevamente la aplicación
- ♦ Entramos a la ruta: <http://localhost:4200/>



Podemos visualizar el nuevo registro:



Conclusión

¡Listo! Con esta lección hemos aprendido a:

- ✓ Crear un componente formulario
- ✓ Integrarlo al sistema de rutas de Angular
- ✓ Usar `ngModel` con `FormsModule`
- ✓ Enviar los datos al backend a través de un servicio
- ✓ Reutilizar el formulario para operaciones futuras como modificar personas

 ¡Nuestro sistema ahora permite agregar nuevos registros de personas desde la interfaz web!

Sigue adelante con tu aprendizaje  , ¡el esfuerzo vale la pena!

¡Saludos! 

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)